Output - 3 2 1 2 3 4

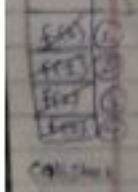
Q1) Predict the Output

```
void main()
int n = 3;
f(n);
}
void f(int n)
if (n==0) return;
cop(n+" - ");
f(n-1);
cop((n+1)+" - ");
}
```

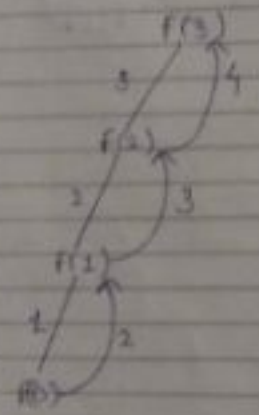
```
void f(int n)
if (n==0)
return;
}
cop(n+" - ");
f(n-1);
cop((n+1)+" - ");
}
```

```
void f(int n)
if (n==0)
return;
}
cop(n+" - ");
f(n-1);
cop((n+1)+" - ");
}
```

```
void f(int n)
if (n==0)
return;
}
```



Recursive Tree



Output:- X X Y Y X Y

Q2]

```
Prum(){
  int n=2;
  f(n);
}
void f(int n){
  if(n==0){
    return;
  }
  sop("X");
  f(n-1);
  sop("Y");
  f(n-1);
}
```

```
void f(int n){
  if(n==0){
    return;
  }
  sop("X");
  f(n-1);
  sop("Y");
  f(n-1);
}
```

```
void f(int n){
  if(n==0){
    return;
  }
}
```

```
void f(int n){
  if(n==0){
    return;
  }
}
```

```
void f(int n){
  if(n==0){
    return;
  }
  sop("X");
  f(n-1);
  sop("Y");
  f(n-1);
}
```

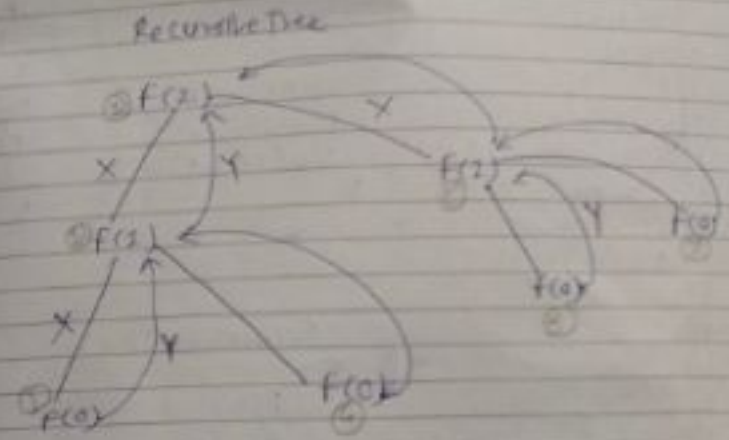
```
void f(int n){
  if(n==0){
    return;
  }
}
```

```
void f(int n){
  if(n==0){
    return;
  }
}
```

Call stack

f(0)	4
f(1)	5
f(0)	6
f(0)	7
f(2)	2
f(1)	1
f(2)	3

Recursive Tree



Output:- 0 2 0 4 0 2 0

Q8

```
Psvm(){
int n=4;
f(n);
}
void f(int n){
if (n<=1){
    sop(n+" ");
    return;
}
f(n-2);
sop(n+" ");
f(n-2);
}
```

```
void f(int n){
if(n<=1){
    sop(n+" - ");
    return;
}
f(n-2);
sop(n+" - ");
f(n-2);
}
```

```
void f(int n){
if (n<=1){
    sop(n+" - ");
    return;
}
```

```
void f(int n){
if(n<=1){
    sop(n+" - ");
    return;
}
```

```
void f(int n){
if(n<=1){
    sop(n+" - ");
    return;
}
f(n-2);
sop(n+" - ");
f(n-2);
}
```

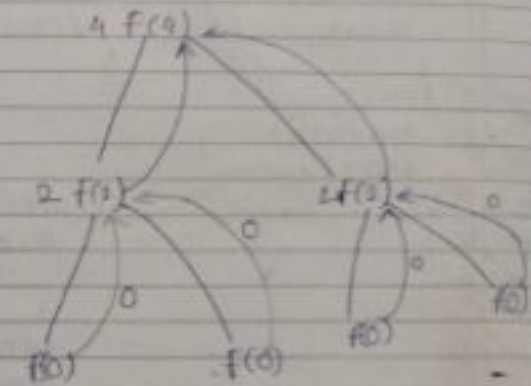
```
void f(int n){
if(n<=1){
    sop(n+" - ");
    return;
}
```

```
void f(int n){
if(n<=1){
    sop(n+" - ");
    return;
}
```

Call stack

f(4)	1
f(2)	2
f(0)	3
f(2)	4
f(0)	5
f(0)	6
f(0)	7

Recursive Tree



Output :- 1 4 2

Q4] main() {
int n=2,
f(n);
}

void f(int n) {
if(n==0) {
return;
}
f(n-1);
cop(n*n+" ");
f(n-1);
}

void f(int n) {
if(n==0) {
return;
}
f(n-1);
cop(n*n+" ");
f(n-1);
}

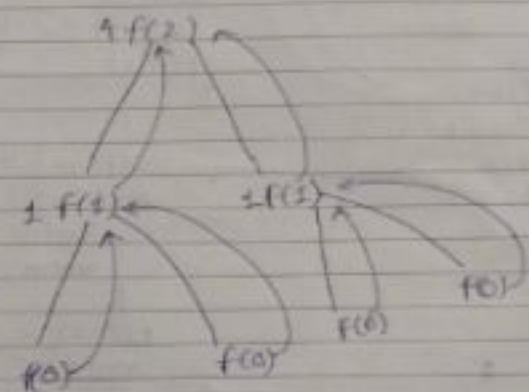
void f(int n) {
if(n==0) {
return;
}
f(n-1);
cop(n*n+" ");
f(n-1);
}

void f(int n) {
if(n==0) {
return;
}
f(n-1);
cop(n*n+" ");
f(n-1);
}

f(0)	6
f(1)	4
f(1)	4
f(2)	2
f(0)	3
f(0)	2
f(1)	1
f(0)	0

Calistack

Recursive Tree



Output :- 5 3 1 1 3 5

Q.5]

```
Pvm(){
int n=5
f(n);
}
void f(int n){
if(n<=0){
return;
}
sop(n+"-");
f(n-2);
sop(n+"-");
}
```

```
void f(int n){
if(n<=0){
return;
}
sop(n+"-");
f(n-2);
sop(n+"-");
}
```

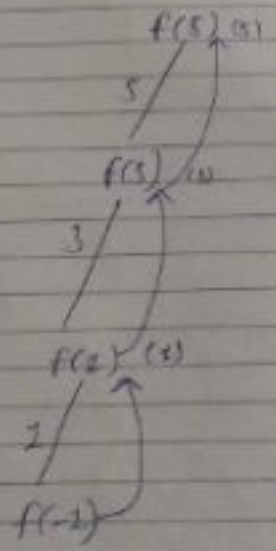
```
void f(int n){
if(n<=0){
return;
}
sop(n+"-");
f(n-2);
sop(n+"-");
}
```

```
void f(int n){
if(n<=0){
return;
}
}
```

~~f(5)~~	2
~~f(3)~~	0
~~f(1)~~	1
~~f(-1)~~	0

Call Stack

Recursive Tree



Output :- 8 4 2 1 2 4 8

Q.9

```
Psvm(){
  int n=8;
  f(n);
}
void f(int n){
  if (n<=1){
    sop(n+"-");
    return;
  }
  sop(n+"-");
  f(n/2);
  sop(n+"-");
}
```

```
Void f(int n){
  if (n<=1){
    sop(n+"-");
    return;
  }
  sop(n+"-");
  f(n/2);
  sop(n+"-");
}
```

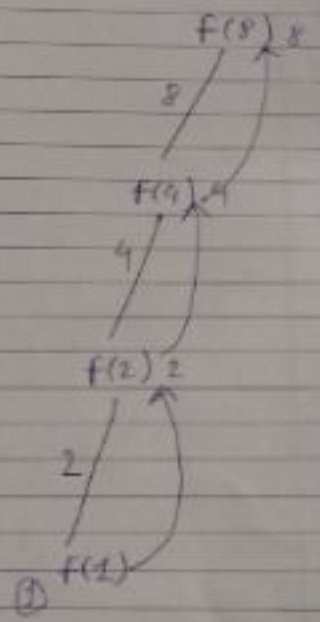
```
void f(int n){
  if (n<=1){
    sop(n+"-");
    return;
  }
  sop(n+"-");
  f(n/2);
  sop(n+"-");
}
```

```
void f(int n){
  if (n<=1){
    sop(n+"-");
    return;
  }
}
```

f(8) ①
f(4) ②
f(2) ③
f(1) ④

Call stack

Recursive Tree



Output : A 3 A 2 A 1 B 0 B 1 B 6

Q1]

```
Prvm(){
  int n=3;
  f(n);
}
void f (int n){
  if (n<=0){
    return;
  }
  sop("A"+n+"_");
  f(n-1);
  sop("B"+(n*2)+"_");
}
```

```
void f (int n){
  if (n<=0){
    return;
  }
  sop("A"+n+"_");
  f(n-1);
  sop("B"+(n*2)+"_");
}
```

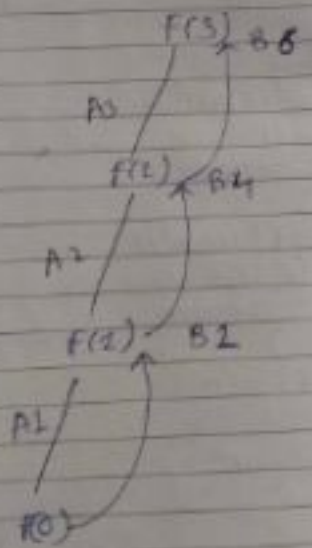
```
void f (int n){
  if (n<=0){
    return;
  }
  sop("A"+n+"_");
  f(n-1);
  sop("B"+(n*2)+"_");
}
```

```
void f (int n){
  if (n<=0){
    return;
  }
}
```

f(0)	2
f(1)	1
f(2)	3
f(3)	4

Callstack

Recursive Tree



Output:- 1 2 3 1 4 1 2

Q3)

```
Prvm(){
int n=4;
f(n);
}
void f(int n){
if (n<=0)
return;
}
f(n-1);
sop(n+" - ");
f(n-2);
}
```

```
```

Output :- 1 2 0 3 1

Q.9]

```
main(){
  int n=3;
  f(n);
}
void f(int n){
  if(n<=1){
    cop(n+" ");
    return;
  }
  f(n-1);
  cop(n+" ");
  f(n-2);
}
```

```
void f(int n){
  if(n<=1){
    cop(n+" - ");
    return;
  }
  f(n-1);
  cop(n+" ");
  f(n-2);
}
```

```
void f(int n){
  if(n<=1){
    cop(n+" - ");
    return;
  }
  f(n-1);
  cop(n+" ");
  f(n-2);
}
```

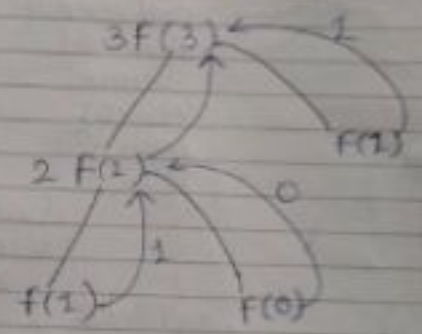
```
void f(int n){
  if(n<=1){
    cop(n+" - ");
    return;
  }
  f(n-1);
  cop(n+" ");
  f(n-2);
}
```

```
void f(int n){
  if(n<=1){
    cop(n+" - ");
    return;
  }
  f(n-1);
  cop(n+" ");
  f(n-2);
}
```

Call Stack

f(1)	5
f(0)	4
f(1)	3
f(2)	2
f(3)	1

Recursive Tree

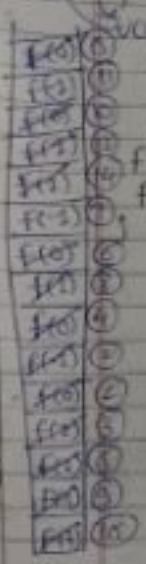


Output :-

Q.10]

```
main(){
    int n=4;
    f(n);
}

void f(int n){
    if(n<=0){
        return;
    }
    f(n-1);
    f(n-2);
}
```



Call Stack

```
void f(int n){
    if(n<=0){
        return;
    }
    f(n-1);
    f(n-2);
}
```

```
void f(int n){
    if(n<=0){
        return;
    }
    f(n-1);
    f(n-2);
}
```

```
void f(int n){
    if(n<=0){
        return;
    }
    f(n-1);
    f(n-2);
}
```

```
void f(int n){
    if(n<=0){
        return;
    }
    f(n-1);
    f(n-2);
}
```

```
void f(int n){
    if(n<=0){
        return;
    }
    f(n-1);
    f(n-2);
}
```

```
void f(int n){
    if(n<=0){
        return;
    }
    f(n-1);
    f(n-2);
}
```

```
void f(int n){
    if(n<=0){
        return;
    }
    f(n-1);
    f(n-2);
}
```

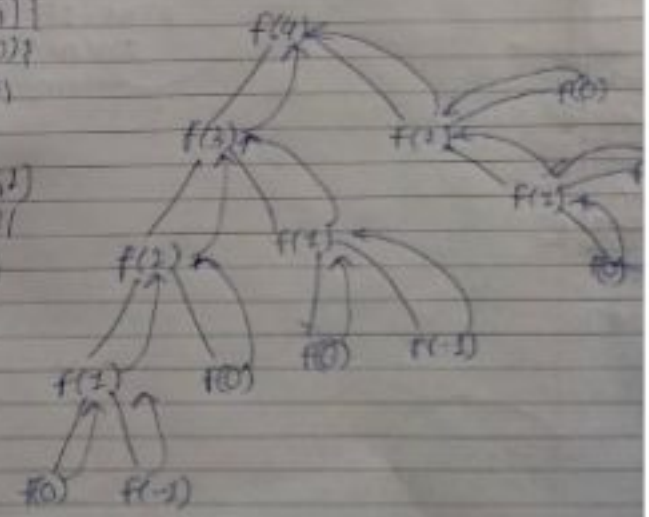
```
void f(int n){
    if(n<=0){
        return;
    }
    f(n-1);
    f(n-2);
}
```

```
void f(int n){
    if(n<=0){
        return;
    }
    f(n-1);
    f(n-2);
}
```

```
void f(int n){
    if(n<=0){
        return;
    }
    f(n-1);
    f(n-2);
}
```

```
void f(int n){
    if(n<=0){
        return;
    }
    f(n-1);
    f(n-2);
}
```

```
void f(int n){
    if(n<=0){
        return;
    }
    f(n-1);
    f(n-2);
}
```

```
void f(int n){
    if(n<=0){
        return;
    }
    f(n-1);
    f(n-2);
}
```

Recursive Tree